

FAST AUTOMATION MODULE MANUAL

Generic single-configuration edition
September 11, 2026

Contents

1	INTRODUCTION	3
2	SYSTEM ARCHITECTURE AND WORKFLOW	3
2.1	Overview	3
2.2	Manual and scheduled operation	4
2.3	Workflow (<code>run.sh</code>)	4
2.4	Directory layout	6
2.5	Logging and alerting	7
3	DEPENDENCIES AND INSTALLATION	7
3.1	Assumed pre-requisites	7
3.2	System-level dependencies	8
3.3	Python environment	8
3.4	Mail delivery (Gmail API)	9
3.5	Installation steps	9
4	CONFIGURATION	10
4.1	Main files	10
4.2	How to read the parameter tables below	11
4.3	<code>globals.var</code>	11
4.3.1	Debug switches	11
4.3.2	Meso-NH / MPI version	12
4.3.3	Base configuration and main paths	12
4.3.4	Logging	12
4.3.5	Init paths	13
4.3.6	Output paths	13
4.3.7	Namelist templates and executables	14
4.3.8	File compression and backup	14
4.4	<code>pgd.var</code>	14
4.5	<code>run.var</code>	15
4.5.1	Simulation parameters	16
4.5.2	Spatial configuration and coordinates	16
4.5.3	Plot parameters	16
4.5.4	DIAG/FICVAL extraction parameters	17
4.6	<code>times.var</code>	18
4.6.1	Date and time	18
4.6.2	Plot parameters	19
4.7	<code>backup.var</code>	19
4.7.1	Upload / backup switches	19

4.7.2	Backup parameters	19
4.7.3	Remote host parameters	20
4.8	filedef.var	20
5	EXECUTABLES REFERENCE	21
5.1	bin/run.sh – main nightly executable	21
5.2	bin/prep_list.sh – generate reference legacy-style scripts	21
5.3	bin/postproc.sh – DIAG/FICVAL extraction	22
5.4	bin/plot_python.sh – plotting	22
5.5	bin/backup.sh – upload and local backup	22
5.6	bin/pgdgen.sh – PGD (orography) generation	23
5.7	Benchmarking helpers (not part of nightly operation)	23
6	GROUND-PARAMETER OUTPUT FILE FORMAT	23
6.1	File naming and header	23
6.2	Processing	24
6.3	Columns	24

1 INTRODUCTION

This manual describes the **automation module** of **FAST** (Forecast Automation System for Telescopes), the component that runs a single, once-per-night Meso-NH / Astro-Meso-NH atmospheric and optical-turbulence forecast for a ground-based telescope site. It automates the full chain: waiting for and validating the initialization files, running the mesoscale simulation, post-processing the raw model output into astroclimatic parameters, generating diagnostic plots, and delivering the results to a remote server.

This document describes the automation *generically*, i.e. independently of the specific telescope, site or forecasting project it is deployed for. FAST has been deployed, in slightly different configurations, for more than one ground-based observatory; this manual covers the common core that any such deployment shares, not the site-specific choices (domain geometry, delivered file formats, calibration constants) that a given installation layers on top of it via its own `conf.d/*.var` files. Where a parameter table below shows a `[current: ...]` value, that value is simply whatever this particular installation's configuration files currently contain, given as a worked example – not a universal default.

This document covers only the *automation* part of FAST, i.e. the component that produces the once-per-night Meso-NH forecast. A separate, complementary *short-timescale* component (an autoregressive or machine-learning correction of the forecast using real-time telemetry, updated every few minutes) may run alongside it on some deployments; it is a distinct program with its own configuration and is documented separately. Section 2 describes the point at which the automation hands data off to it, where applicable.

The user of this manual is assumed to be familiar with the basic operation of Meso-NH, with the Linux shell, and with basic bash scripting. A working installation of Meso-NH and of a local OpenMPI build is assumed to already be in place; Section 3 lists the remaining dependencies and the steps needed to install and configure the automation code on top of them.

Scope of this revision. This is a *single-configuration* edition of the manual: it documents one self-contained instance of the automation (one `bin/` tree, one set of `conf.d/*.var` files, one nightly `run.sh`), started directly (by cron or manually) with no external orchestration layer coordinating multiple instances or deciding which one's output to keep. Deployments that do chain several configurations together (for example, running a second, different- resolution configuration after the first, or driving several instances from a shared top-level script) require that coordination layer to be documented separately, on top of this manual; nothing here describes it.

A note on scope. This manual documents what the automation does and how it is configured; it assumes solid Linux system administration and shell-scripting skills, and it is not a substitute for them. In an operational deployment, reading the source code directly – not just this manual – is eventually unavoidable: some situation not covered here will arise, and resolving it will require understanding the scripts themselves. No manual can substitute for the operational experience needed to run an unattended automated system in production; that experience is built on the job, by encountering and working through real problems as they occur.

2 SYSTEM ARCHITECTURE AND WORKFLOW

2.1 Overview

This edition of the automation runs **one Meso-NH configuration, once per night**: a single set of nested domains (`NESTING` in `globals.var`, currently 3 – outer domain plus two nested levels), driven by one self-contained `bin/` tree and one `bin/conf.d/*.var` configuration (Section 4). There is no built-in mechanism in this codebase for running several configurations in sequence

or for having one run’s output conditionally replace another’s; if a deployment needs that (e.g. a coarser and a finer-resolution configuration chained together), it has to be implemented as an external wrapper around this code, and documented separately.

2.2 Manual and scheduled operation

The whole procedure is a single entry point, `bin/run.sh`, invoked with no arguments:

```
cd bin
./run.sh
```

For nightly unattended operation this is normally launched from cron, e.g.:

```
0 20 * * * /path/to/bin/run.sh > /path/to/sim.out 2>&1
```

`run.sh` locates its own configuration via `conf.d/globals.var` relative to its own path, so it must be run from within `bin/` (or by an absolute path that still resolves correctly). The date being processed is controlled by `YEAR/MONTH/DAY` in `times.var` (Section 4): under normal operation these are meant to track “today” (they default to the system date at the top of the file), but **a hand-set override further down the file currently takes precedence** – see the note in Section 4 on `times.var`. To process a specific night (whether for the scheduled run or for manual reprocessing of a past date), edit that override directly. There is no external process that substitutes today’s date into the configuration automatically; whatever `times.var` contains at the time `run.sh` sources it is what gets used.

2.3 Workflow (`run.sh`)

`bin/run.sh` performs, in order:

1. **Startup:** kills any leftover Meso-NH-related processes from a previous, possibly stuck run (`pkill` on the executable names from `globals.var`), rotates the log directory, and sources `times.var` then `run.var`.
2. **Initialization file retrieval:** sources `filedef.var` directly – **this is where the actual wait happens**, in `run.sh`’s own process. `filedef.var`’s logic waits for the raw initialization files for the requested forcing hours to appear in `INIRAWDIR`, retries downloading them from a configurable backup server over SSH if they are late or missing, verifies each file’s MD5 checksum, and merges the orography information from the analysis file into each forecast file with `grib_copy` – aborting with an alert mail if the files still have not arrived after a configurable timeout (`TIMELIMIT` in `times.var`). `run.sh` then sources `backup.var`.
3. **Reference script generation:** only once the previous step has confirmed the initialization files are present, and only if `TESTRUN=0` and `PREPLISTTRUE=1` (and `NOPREP=0/NORUN=0`), `run.sh` invokes `bin/prep_list.sh` as a sub-process. As explained in full in Section 5, `prep_list.sh`’s own purpose is to generate a pair of self-contained, legacy-style reference scripts, not to fetch files; it re-sources `filedef.var` itself (in its own, separate shell) purely to know which files to write into those scripts, and since they are already present by this point, this second sourcing does not itself wait on anything. `run.sh` then moves the two generated scripts into `LOGDIR`.
4. **PREP_REAL phase:** for every nested domain, runs Meso-NH’s `PREP_REAL_CASE` (and `SPAWNING` for nested domains) to build the initial and boundary conditions.

5. **EXE phase:** runs the MESONH executable under `mpirun`, producing the raw `.lfi` model output for the whole forecast segment.

6. **POSTPROC phase** (`bin/postproc.sh`, sourced): runs the `DIAG` step and `conv2dia/diaprolog` to derive the astroclimatic parameters (optical turbulence C_N^2 and integrated parameters, wind, temperature, relative humidity, PWV, etc.) and to extract them as ASCII (FICVAL vertical profiles/cuts and 2D ground maps) at the points, domains and heights configured in `run.var`.

7. **PLOT phase** (`bin/plot_python.sh`, sourced): runs the Python plotting suite under `bin/conf.d/Utils/Plot_python/` (one script per parameter/plot type) to turn the ASCII output into the delivered figures.

8. **Compression:** optionally `bzip2`-compresses the raw `PREP_REAL`, `RUN` and `DIAG .lfi` outputs in parallel (GNU `parallel`), controlled by `DOZIPREAL/DOZIPOUT/DOZIPDIAG` in `globals.var`.

9. **Move to staging:** moves the outputs selected for preservation (`BACKUP_REAL/BACKUP_RUN/BACKUP_DIAG/BACKUP_POSTPROC/BACKUP_LOG` switches in `backup.var`) into a per-run staging directory (`PREPARE_DIR`); everything else is deleted.

10. **Upload/backup** (`bin/backup.sh`, sourced, unless deferred with `BACKUPLATER`): `rsyncs` the figures and ASCII ground outputs to the configured remote server(s) (Section 4, `backup.var`), and/or keeps a local copy, according to `backup.var`.

11. **Notification and hand-off:** sends a completion e-mail with a per-phase timing summary; writes the day's configuration file for the short-timescale component into the `config/` subdirectory of `AUTOREGRESSION/`, and `rsyncs` the whole `AUTOREGRESSION/` directory to the configured hand-off host(s), if that component is in use on this deployment (Section 3). If so, the automation run must complete before it can operate correctly for that night.

Figure 1 summarizes this sequence.

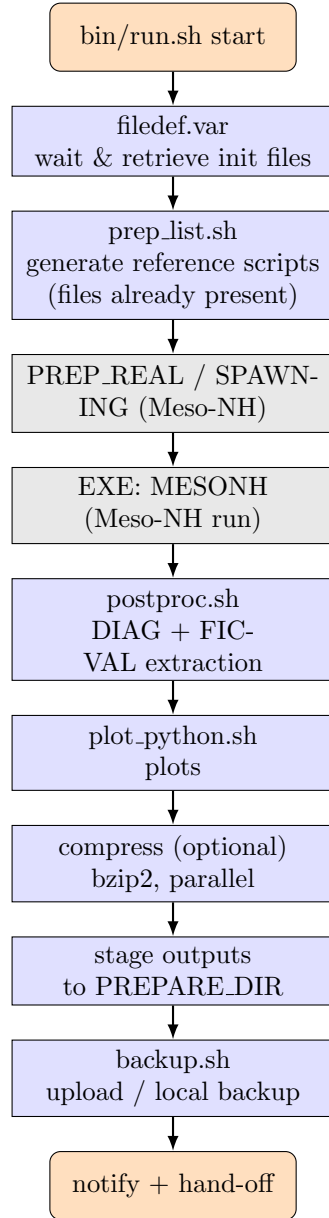


Figure 1: Internal phase sequence of `bin/run.sh`. Grey boxes are Meso-NH itself; blue boxes are automation scripts.

Each of these phases can be individually disabled for debugging via the `NOPREP/NORUN/NODIAG/NOFICVAL/NOLOT` switches in `globals.var`; this is intended for manual troubleshooting only.

`bin/pgdgen.sh` is a separate, standalone script (not called by `run.sh`) used to (re-)generate the static PGD orography files for a site; it is normally run once, at installation time (Section 3), and only re-run if the domain geometry or the ground/topography input files change.

The scripts `bin/Run_list.sh`, `bin/crossc2d.py` and `bin/multi.py` are CPU/timing benchmarking helpers used to test the code against a list of past initialization dates; they are not part of the nightly operational workflow.

2.4 Directory layout

The automation lives in one top-level working directory (`WORKDIR` in `globals.var`):

bin/	: scripts (see Section 5)
bin/conf.d/	: configuration files, namelist templates, correction tables, compiled fortran helpers (see Section 4)
PGD/	: static orography files, generated once by pgdgen.sh and persistent across runs

The following sub-directories are created fresh by each run and removed/replaced at the end of it (anything left behind after a failed run is there for post-mortem debugging only, and will be deleted on the next successful run):

REAL/	: PREP_REAL binary output and namelists
RUN/	: Meso-NH (EXE) binary output and namelists
DIAG/	: DIAG / conv2dia binary output and namelists
ASCII/	: ASCII output of the postprocessing (DAT, FICVAL, PLOT_DAT, AUTOREGRESSION sub-directories)
PLOT/	: figures produced by the Python plotting suite
LOG/	: logs of the current/last run

Outside WORKDIR, in the automation user's home directory, the following are used across runs:

<init dir>/	: combined (orography + forecast) init files for the current night (INIDIR)
<backup init cache>/	: local cache of files retrieved from the backup init server (BKSERVER_LOCALPATH)
BACKUP_OUTPUT/	: per-night backup of ASCII/plot outputs (see backup.var)
BACKUP_FIGURES/	: local mirror of the delivered figures
<staging dir>/	: staging directory used while moving outputs (PREPARE_DIR)
AUTOREGRESSION/	: hand-off area shared with the short-timescale component, if in use on this deployment
TOPOGRAPHY/	: source topography files used to build the PGD
<python virtualenv>/	: Python virtualenv used by the automation (Section 3)

The exact names in angle brackets above are whatever `globals.var` currently sets them to (Section 4); they are not fixed by the code.

2.5 Logging and alerting

Each phase writes to its own timestamped log file under LOGDIR (main log, error log, and one log per Meso-NH executable), configured in `globals.var`. A cumulative log recording start/end time and duration of every phase of every run is appended to the path set by `LOG_TIME_CUMULATIVE` in `globals.var`. Fatal problems abort the script and send an alert e-mail (function `error`); non-fatal issues are logged and mailed without stopping the run (function `error_soft`).

3 DEPENDENCIES AND INSTALLATION

3.1 Assumed pre-requisites

This section assumes that **Meso-NH** and a matching local **OpenMPI** build are already installed and working. The codebase currently in use expects Meso-NH **V5.2.1** built against **eccodes 2.30.0**, set in `globals.var` as:

```
MNH_BASE_PATH="${HOME}/MNH-V5-2-1_eccodes_2_30_0"
```

and OpenMPI 4.1.1. A custom Meso-NH VER_USER build is required; which build is active is selected by MESONH_PERSO_VERSION in `globals.var` (Section 4). On this deployment it switches seasonally (winter/summer) between two builds, because the optical- turbulence calibration built into the model is itself seasonal and the switch is driven by the current date; a fixed, non-seasonal build can also be used (see the note in Section 4).

A note on Meso-NH output format. This codebase reads and writes Meso-NH's legacy `.lfi` binary output format throughout (PREP_REAL, EXE and DIAG outputs, the optional `bzip2` compression step, the `conv2dia/diaprog` post-processing). It has **not** been adapted to the `.nc4`/NetCDF-4 output format used by more recent Meso-NH versions; deploying it against a Meso-NH build that only produces `.nc4` output will require changes to the post-processing chain that are outside the scope of this manual.

3.2 System-level dependencies

PROGRAM	NOTES
Meso-NH	V5.2.1, custom VER_USER build (see above)
OpenMPI	4.1.1
eccodes (incl. <code>grib_copy</code>)	2.30.0, used to merge orography into forecast grib files
gfortran + <code>make</code>	used to build the helper programs in <code>bin/conf.d/utls</code>
bash	$\geq 4.x$
Python 3	used both directly and inside a dedicated virtualenv (below)
GNU <code>parallel</code>	parallel <code>bzip2</code> compression of Meso-NH output
ImageMagick (<code>convert</code>)	format conversion in the plotting phase
<code>bc</code>	floating point arithmetic in the bash scripts
OpenSSH client (<code>ssh/scp</code>)	retrieval from the backup init server, uploads
<code>rsync</code>	delivery of figures/ASCII outputs and hand-off to the AR hosts

3.3 Python environment

The automation code expects a Python virtualenv at a fixed path referenced by `LOCPYTHONDIR` in `globals.var`; `run.sh` activates it at the start of every run and deactivates it at the end. If a short-timescale component is also installed on the same host, it can share this same virtualenv.

The packages required by the automation scripts themselves (plotting, ephemerides, mail) are:

```
numpy
scipy
matplotlib
pytz
python-dateutil
ephem
google-api-python-client
google-auth
```

If the same virtualenv is also used to run a short-timescale/AR component, check that component's own environment manual for any additional version constraints; they are not needed by the automation code on its own.

To create the environment from scratch:

```
cd $HOME
python3 -m venv <venv_name>
source $HOME/<venv_name>/bin/activate
```

```
pip install --upgrade pip
pip install numpy scipy matplotlib pytz python-dateutil ephem \
    google-api-python-client google-auth
deactivate
```

(replace `<venv_name>` with whatever `LOCPYTHONDIR` in `globals.var` points at).

3.4 Mail delivery (Gmail API)

Alert and completion e-mails are sent through the Gmail API rather than through a local MTA, via `sendmail_gmail.py` under `bin/conf.d/utils/` – this is the script actually wired into every alert path, through `common_functions.func`. It expects an OAuth client secret and a cached token in the Python virtualenv directory:

```
<venv_name>/credentials.json : OAuth 2.0 client secret (download from the Google Cloud
                             project used for this mail account)
<venv_name>/token.pickle    : cached, auto-refreshed OAuth token; generated on first
                             successful authentication
```

A same-named `sendmail_attach.py` also exists under `bin/conf.d/utils`, using a plain `smtplib`/SMTP implementation instead, but it is not currently invoked anywhere in this codebase; it can be ignored for installation purposes. Setting up `credentials.json` and performing the first interactive authorization (to create `token.pickle`) must be done once, outside of the automated procedure, before the first scheduled run; afterwards the token is refreshed automatically. Without valid credentials the scripts abort rather than silently failing to notify.

3.5 Installation steps

1. **Unpack the code.** Extract the automation directory to its intended location (it can live anywhere, but conventionally directly under the automation user's home directory).
2. **Point the configuration at its working directory.** In `bin/conf.d/globals.var`, set `WORKDIR` to the absolute path of that directory.
3. **Build the fortran helper programs.** These are not portable binaries and must be recompiled on the target machine:

```
cd bin/conf.d/utils
./Make_All.sh
```

This builds every helper under a `source_*` sub-directory (post-processing and correction tools used by `postproc.sh`/`plot_python.sh`); the script aborts on the first compilation failure, so a clean run confirms the whole helper suite is usable.

4. **Set up the Python virtualenv and the Gmail API credentials** as described above.
5. **Set up passwordless SSH (key-based)** from the automation account to three groups of remote hosts. First, the backup initialization server (`BKSERVER_USER`/`BKSERVER_IP` in `globals.var`). Second, the primary and backup output servers (`REMOTE_USER`/`REMOTE_HOST` and `REMOTE_USER_BK`/`REMOTE_HOST_BK`, both in `backup.var`). Third, if a short-timescale component is deployed, its hand-off host(s) (`REMOTE_AR`/`REMOTE_AR_BK`, also in `backup.var`).
6. **Generate the PGD (orography) files.** Once the topography source files are in place (`TOPODIR` in `globals.var`) and `pgd.var` is configured for the site (Section 4):

```
cd bin
./pgdgen.sh
```

A successful run populates PGD/. This step only needs to be repeated if the domain geometry, nesting configuration or ground/topography input files change; it is otherwise a one-time step per site.

7. **Do a manual test run** before relying on cron:

```
cd bin
./run.sh
```

Inspect LOG/ for errors; the most common causes of failure at this stage are a missing/incorrect WORKDIR, missing PGD files, an un-built helper program, or missing SSH/API credentials.

8. **Install the cron entry** shown in Section 2 for nightly operation.

4 CONFIGURATION

4.1 Main files

Inside the working directory tree, the executables and configuration files that a user is expected to interact with are the following:

bin/	{	run.sh	:main nightly executable
		pgdgen.sh	:one-time PGD (orography) generation
		prep_list.sh	:generates reference legacy-style scripts, called by run.sh
		postproc.sh	:DIAG/FICVAL extraction, sourced by run.sh
		backup.sh	:upload/backup, sourced by run.sh
		plot_python.sh	:plotting, sourced by run.sh
bin/conf.d/	{	Run_lista.sh	:Run in postprocessing on a sequence of dates
		globals.var	:main configuration file (paths, logging, MNH/MPI setup)
		pgd.var	:orography/PGD generation configuration
		run.var	:simulation, spatial and DIAG/plot extraction parameters
		times.var	:date, time and plot-window configuration
		filedef.var	:init file naming/retrieval logic (mostly scripted)
		backup.var	:upload and backup configuration

Other files under bin/ and bin/conf.d/ not listed here (and not described in Section 5) are internal helpers, legacy variants or testing scripts; they should not normally need to be touched.

Further sub-directories of bin/conf.d/ are used as follows:

```
bin/conf.d/namelist/cen4th/ : Meso-NH namelist templates actually used by this
                             configuration (centred 4th-order advection scheme;
                             referenced by NLDIR in globals.var)
bin/conf.d/namelist/weno3/  : an alternative (WENO 3rd-order advection scheme)
                             namelist set, physically present but not referenced
                             by any current .var file -- a leftover from a
                             different configuration this code was derived from
```

Most Meso-NH namelist parameters are set from `run.var`/`times.var` at run time; the namelist templates only need to be edited directly for parameters that are not exposed as `.var` variables. This should only be done by someone familiar with Meso-NH namelists.

NOTE CAREFULLY: namelist templates are edited through keyword substitution. All strings that end with `_VALUE` or `_FILE` are typically edited automatically. Do not change them if you do not want to make automatic substitution to silently fail.

<code>bin/conf.d/corrections/</code>	: per-parameter multiplicative/additive calibration tables, applied during postprocessing (<code>plot_python.sh</code>)
--------------------------------------	---

Each file is a small ASCII table: one line of `MULT SUM`, or of `MULT SUM START END` for time-windowed corrections. Files are named after the parameter they calibrate (e.g. `pwv_integ.var`, `tau_integ.var`), with some parameters also carrying seasonal variants. Several parameters have both a default table and an `_OS18` variant; both are actively used by `plot_python.sh` to compute an alternative (“OS18”) product – for seeing, isoplanatic angle and coherence time – alongside the default one; see the note on `plot_python.sh` in Section 5.

<code>bin/conf.d/utils/</code>	: compiled fortran helper programs and their sources (<code>source_*</code> sub-directories, built by <code>Make_All.sh</code> , see Section 3), the Python plotting suite (<code>Plot_python/</code>), the Gmail-API mail senders, small ephemeris helpers (<code>Sunset.py</code> , <code>Sunrise.py</code> , <code>Dawn.py</code> , <code>Dusk.py</code>) and <code>common_functions.func</code> (the bash functions <code>-- log</code> , <code>error</code> , <code>error_soft</code> , <code>check_directory</code> , etc. <code>-- sourced</code> by every script in <code>bin/</code>)
--------------------------------	---

4.2 How to read the parameter tables below

For each configuration file, parameters are listed with a short description and their current value, shown as `[current: ...]` – i.e. whatever this particular installation’s configuration files happen to contain today, given purely as a worked example of the kind of value expected, not as a recommended or universal default. A variable not documented below is either a purely internal/scripted variable (computed from other variables, not meant to be edited), a debug switch, or an undocumented/experimental feature – such variables should only be changed by someone who has read the corresponding script.

4.3 `globals.var`

`globals.var` holds the global configuration: debug switches, the Meso-NH/MPI/Python setup, the main working paths, logging and the init/output path tree. It is sourced first by every script.

4.3.1 Debug switches

<code>TESTRUN=0/1</code>	: if 1, do not wait for real init files; use whatever init files are manually specified/present instead. See section~\ref{sec:filedef} to understand how it works. <code>[current: 0]</code>
<code>PREPLISTTRUE=0/1</code>	: if 1, <code>run.sh</code> invokes <code>prep_list.sh</code> at the start of the run. <code>[current: 1]</code>
<code>WAITSURE=0/1</code>	: if 1, wait one extra minute after all init files are detected, to be extra sure the upload has really finished. <code>[current: 0]</code>

PARSEBACKUP=0/1 : internal use only, must be left at 0.
 BACKUPLATER=0/1 : if 1, backup.sh is not sourced automatically at the end of run.sh
 (it must then be run separately by hand or by an external wrapper
 script, which this codebase does not provide). [current: 0]
 NOPREP/NORUN/
 NODIAG/NOFICVAL/
 NOPLOT=0/1 : debug switches to individually skip the PREP_REAL, EXE, DIAG,
 FICVAL-extraction or PLOT phase. All [current: 0]; only meant for
 manual troubleshooting (see the workflow description above).

4.3.2 Meso-NH / MPI version

SHORTVERSION="string" : short tag for the Meso-NH version, used in log/output file
 names. [current: "M521"]
 MNH_BASE_PATH="string" : root of the Meso-NH installation.
 [current: "\${HOME}/MNH-V5-2-1_eccodes_2_30_0"]
 MESONH_PERSO_VERSION="string": name of the custom Meso-NH VER_USER build to use. On
 this deployment it is computed automatically from the current
 month (a "winter" build for Oct-Mar, a "summer" build for
 Apr-Sep), each hard-coded as a commented-out fallback
 just above the seasonal if/else, so a fixed build can be
 forced by uncommenting one and removing the seasonal logic. The
 winter/summer split is a seasonal calibration of the
 optical-turbulence model built into that version, not a
 change of advection scheme -- see Section 3.1.
 MPI_VERSION="string" : root of the local OpenMPI installation.
 [current: "\${HOME}/OPEN-MPI-4-1-1"]

MNH_MAIN_PATH, MNH_GRIBAPI_PATH, MNH_TOOL_PATH, MNH_PROFILE and the LD_LIBRARY_PATH/
 PATH exports below them are derived automatically from the three variables above and from
 Meso-NH's own profile.mesonh; they should only need to change together with a Meso-
 NH/eccodes version upgrade.

4.3.3 Base configuration and main paths

SITE_NAME="string" : max 3 characters, identifies the site. [current: 3-letter site
 tag]
 NESTING=integer : number of nested domains (outer=1). [current: 3]
 WORKDIR="string" : absolute path of the automation's working directory. Must be
 edited at install time (see the Installation steps above).
 LOCPYTHONDIR="string": bin/ directory of the Python virtualenv activated by run.sh.
 PYTHONEXE="string" : Python interpreter name. [current: "python3"]

CONFDIR, EXEDIR, UTILDIR, PYTHONPLOT, FUNCFILE and the *_VAR variables pointing at the
 other configuration files (FILEDEF_VAR, TIME_VAR, TIME_VAR_SEQ, RUN_VAR, PGD_VAR, BACKUP_VAR,
 CORRECTION_DIR) are all derived from WORKDIR and should not normally be edited.

4.3.4 Logging

```

LOGEXE=0/1/2      : 0 prints MESONH output to the console; 1 sends it to separate log
                   files; 2 appends it to the main log file. [current: 1]
MAIL_ADDRESS="string": destination address(es) for alert/completion mail.
FROM_ADDRESS="string": sender address used by the Gmail-API mail scripts.
MAIL_PREFIX="string" : subject-line prefix for all mail from this deployment. Built as
                   "<project tag> - ${NESTING}DOM"; the checked-in value still carries
                   a
                   project tag inherited from the codebase this automation was derived
                   from -- edit it to identify your own deployment.

```

LOGDIR and every LOG*FILE variable are timestamped paths derived from WORKDIR/DATESTART; LOG_TIME_CUMULATIVE and ERROR_OUTPUT_FILE live outside WORKDIR, directly under \$HOME, so that they survive across runs (the cumulative timing log and the hard-error marker file, respectively). **Note:** the comment immediately above ERROR_OUTPUT_FILE in the checked-in file still warns to also update an external orchestration script if this variable is renamed; that script is not part of this codebase (see the note on scope in Section 2) and the comment is stale – safe to ignore, or to remove, on this single-configuration deployment.

4.3.5 Init paths

```

INIRAWDIR="string" : directory where raw initialization files are uploaded by the
                   forecast-model data provider.
INIDIR="string"    : directory where the merged (orography+forecast) files are
                   placed, by the retrieval logic in fileddef.var.
BKSERVER_USER="string" : SSH username on the backup init server. Requires
                   passwordless key-based login (see the Installation steps).
BKSERVER_IP="string"  : IP of the backup init server.
BKSERVER_INIPATH="string": remote path (relative) of the init files on that server.
BKSERVER_LOCALPATH="string": local cache directory for files retrieved from it.

```

4.3.6 Output paths

The remaining paths in this block are almost all mechanically derived from WORKDIR or \$HOME and are not normally edited by hand; they are listed here for completeness since they determine where each phase’s output can be found (Section 2, “Directory layout”):

Variable	Purpose
OUTDIR_REAL / OUTDIR_PREP	PREP_REAL binary output and namelists (WORKDIR/REAL)
OUTPUT_MNH / PREP_MNH	EXE (Meso-NH run) binary output and namelists (WORKDIR/RUN)
DIAG_MNH / PREP_DIAG	DIAG/conv2dia binary output and namelists (WORKDIR/DIAG)
ASCII_DIR, DAT_DIR, PLOTDIR, FIC-VAL_DIR	ASCII postprocessing output (WORKDIR/ASCII/...)
AUTOREGRESSION_DIR	per-run ASCII staged for the short-timescale hand-off (ASCII_DIR/AUTOREGRESSION)
PLOT_DIR	figures produced by the plotting phase (WORKDIR/PLOT)
AUTOREGRESSION_STORE_DIR	final hand-off data store, a site-named subdirectory under \$HOME/AUTOREGRESSION
AUTOREGRESSION_BASE_DIR	hand-off root, rsynced to the short-timescale host(s) (\$HOME/AUTOREGRESSION)
TRENDSDIR	trend files (\$HOME/TRENDS)
FINALOUTDIR	final ground-parameter output file, delivered to the telescope infrastructure; path under \$HOME is deployment-specific
PREPARE_DIR	per-run staging directory before backup/upload; path under \$HOME is deployment-specific
PGDDIR / PREP_PGDDIR	PGD files (WORKDIR/PGD)
TOPODIR	source topography files (\$HOME/TOPOGRAPHY)
FILTERDIR	short-timescale-component working files (\$HOME/FILTER)

As with MAIL_PREFIX above, several of these paths still carry, in the checked-in file, naming inherited from the codebase’s origin – among them INIDIR, LOCPYTHONDIR, the autoregression store path, FINALOUTDIR, PREPARE_DIR, and the backup-server and raw-init-directory variables above. Rename them freely to fit your own deployment; nothing else depends on the literal strings, only on the variables pointing at consistent, existing directories.

4.3.7 Namelist templates and executables

NLDIR and the NL* variables point at the namelist template files under bin/conf.d/namelist/ (Section 4.1); on this deployment NLDIR points at the cen4th set. The EXE_*/GRIBCOPY variables are the paths to the Meso-NH executables and to grib_copy, derived from the Meso-NH paths above; they should not need manual editing on a standard installation.

4.3.8 File compression and backup

```
DOZIPREAL=0/1 : bzip2-compress the PREP_REAL .lfi output. [current: 0]
DOZIPOUT=0/1 : bzip2-compress the EXE (Meso-NH run) .lfi output. [current: 0]
DOZIPDIAG=0/1 : bzip2-compress the DIAG .lfi output. [current: 0]
```

4.4 pgd.var

pgd.var configures the generation of the static PGD (orography) files by pgdgen.sh (Section 3). On this deployment it describes a domain hierarchy of up to 4 nested levels, even though

NESTING (in `globals.var`) is currently 3: it is NESTING that determines how many of those levels `pgdgen.sh` actually generates, so the file can safely carry parameters for one more level than is presently used, ready for a future domain to be enabled.

```

PGDEXTRACTPT=0/1 : extract latitude/longitude/altitude from the PGD files (only useful
                    if NESTING>1). [current: 1]
PGD_PREFIX="string": 3-character site prefix for generated file names.
                    [current: "${SITE_NAME}"]
XRPK_VALUE=real : projection type: 0=Mercator, 1=north polar, -1=south polar (write as
                    a real number, e.g. "1." not "1"). [current: "0."]
YCOVER_FILE="string": ground cover file (no extension). [current:
                    "ECOCLIMAP_II_EUROP_V2.3"]
YCLAY_FILE="string" : clay file (no extension). [current: "CLAY_HWSO_MOY"]
YSAND_FILE="string" : sand file (no extension). [current: "SAND_HWSO_MOY"]
XLATCEN_VALUE=real : latitude of the domain centre (site-specific).
XLONCEN_VALUE=real : longitude of the domain centre (site-specific).
NIMAX_VALUE=integer : outer-domain grid points along X. [current: "80"]
NJMAX_VALUE=integer : outer-domain grid points along Y. [current: "${NIMAX_VALUE}"]
XDX_VALUE=meters : outer-domain grid spacing along X. [current: "10000"]
XDY_VALUE=meters : outer-domain grid spacing along Y. [current: "${XDX_VALUE}"]

```

The following are bash arrays. `YZS_FILE/YFILETYPE_VALUE` have one element per domain *including* the outer one (element 0 = outer domain, element 1 = first nested domain, and so on); the other four arrays below have one element per *nested* domain only, since the outer domain has no parent to be positioned/scaled against (element 0 = first nested domain, i.e. the second domain overall, element 1 = second nested domain, and so on):

```

YZS_FILE[n]="string" : topography source file (no extension) for domain n+1 (n=0
                        is the outer domain). Global coverage datasets (e.g.
                        "gtopo30") are typically used for the outer levels, and a
                        finer, site-specific high-resolution dataset for the
                        innermost nested level(s) -- named after the site on this
                        deployment.
YFILETYPE_VALUE[n]="string" : Meso-NH file-type keyword matching YZS_FILE[n] (e.g.
    DIRECT,
                                LATLON). [current: "DIRECT" for all domains]
IXOR_PREC[n], IYOR_PREC[n] : starting grid point (X,Y) of nested domain n+2,
    expressed on
                                its parent (preceding) domain grid.
IXSIZE_VALUE[n],
IYSIZE_VALUE[n] : number of grid points of nested domain n+2.
IDXRATIO_VALUE[n],
IDYRATIO_VALUE[n] : grid-refinement ratio of nested domain n+2 with respect to
                    its parent domain.

```

These last four arrays only need to change if the domain geometry itself changes (position, extent or resolution of a nested domain); doing so requires re-running `pgdgen.sh`.

4.5 run.var

`run.var` holds the Meso-NH simulation parameters proper (timestep, nesting, microphysics, domain geometry), the plotting height/limit parameters, and the configuration of the DIAG/FICVAL extraction (which parameters, at which points/domains/heights, are exported as ASCII for plotting and for the short-timescale hand-off). Most variable names mirror the corresponding Meso-NH namelist keyword. This file must be sourced after `times.var`.

4.5.1 Simulation parameters

NUM_PROC=integer : number of MPI processes for the simulation. [current: 64]
NAM_SIMU="string" : 5-character CEXP (Meso-NH experiment name). [current: "\${DAY}_\${MONTH}"]
NAM_SEG="string" : 5-character CSEG (Meso-NH segment name). [current: "y\${YEAR}"]
NMODEL_VAR=integer : number of models/domains, normally equal to NESTING.
XTSTEP_VAR=seconds : outermost-domain Meso-NH timestep. [current: 12]
NDTRATIO_ARRAY[n] : timestep ratio of nested level n+1 with respect to its parent (element [0]=1 always). [current: 1,2,3,4]
NDAD_ARRAY[n] : parent-level index of nested level n+1 (element [0]=1 always). [current: 1,1,2,3]
XRIMKMAX[n] : maximum lateral-relaxation coefficient (1/s) for nested level n+1; kept hand-tuned rather than auto-computed (a commented-out auto-generation formula is kept in the file for reference, but is not used -- it was found to lead to instability). [current: 0.0006666, 0.0020000, 0.0013333]
CCLOUD_VAR="string": microphysical scheme. [current: "KESS"]
CPRESOPT_VALUE="string": pressure solver. [current: "ZRESI"]
NUMOUTTIMES=integer: number of Meso-NH (.lfi) output times. [current: 15]
OUTTIMESTEP=seconds: interval between outputs; defaults to EXITSEC from times.var -- changing one without the other causes inconsistencies.

XSEGLEN (total simulated seconds) and the XFMOU[] array (the list of output times themselves) are computed automatically from NUMOUTTIMES and OUTTIMESTEP; they should not be edited directly.

4.5.2 Spatial configuration and coordinates

XSNG1="string", YSNG1="string": grid coordinates (I,J) of the first extraction point (p1.dat). [current: "61","61"]
XSNG2="string", YSNG2="string": grid coordinates (I,J) of the second extraction point (p2.dat). [current: "60","61"]
SEESTPOINT="string" : starting point index for seeing integration (see_p1_v.dat). [current: "2"]
DOM_FIRST="string" : nesting level of the first extraction point (1=outer). [current: "3"]
DOM_SECOND="string" : nesting level of the second extraction point. [current: "3"]
SUPCOMP_VAR="F" : enables supplementary computations; always "F" in operation.
MESODIAG_VAR="F" : "F"=normal run, "T"=DIAG-only mode (internal use).
NKMAX_VALUE="string": number of vertical levels. [current: "54"]
ZDZGRD_VALUE="string": near-ground vertical mesh size (m). [current: "20"]
ZDZTOP_VALUE="string": near-top vertical mesh size (m). [current: "600"]
ZZMAX_STRGRD_VALUE="string": altitude (m) separating the two constant-stretching layers. [current: "3500"]
ZSTRGRD_VALUE="string": constant stretching (%) in the lower layer. [current: "20"]
ZSTRTOP_VALUE="string": constant stretching (%) in the upper layer. [current: "00"]

4.5.3 Plot parameters

PLOTBASEH1, PLOTBASEH2 : lower plot height limits (m). [current: 20, 600]
PLOTTOPTH1, PLOTTOPTH2 : upper plot height limits (m). [current: 600, 20000]

CN2H2=meters : intermediate height limit for CN2/wind/RH plots. [current: 2000]
 MRH2=meters : intermediate height limit for mixing-ratio plots. [current: 5000]
 CN2_WRESALE_H=meters : height above which the wind-gradient correction is applied to CN2 profiles. [current: 700]
 CN2_ALPHA, CN2_BETA : coefficients of that correction. [current: 1., 1.5]
 DOM_FIRST_DX,
 DOM_SECOND_DX=km : grid spacing used for 2D maps (must match the PGD grid). [current: 0.5, 0.5]
 WINDSPEEDHIGHLIM=m/s : threshold on the average near-ground (K=4) wind speed, above which backup.sh still uploads the wind figures/maps even when full-figure upload is otherwise disabled -- see Section~\ref{sec:usage}. [current: 8.5]

Changes to this block generally require matching manual edits to `plot_python.sh`.

4.5.4 DIAG/FICVAL extraction parameters

These control which parameters are exported by the DIAG/FICVAL phase (`postproc.sh`), and are also the parameters whose file layout `plot_python.sh` depends on – changing them may require matching edits there.

EXTRACTLIST_DOM_FIRST,
 EXTRACTLIST_DOM_SECOND="string": space-separated list of diaprog keywords for 2D ground maps / point extraction, for DOM_FIRST and DOM_SECOND respectively. [current: DOM_FIRST carries the full list (wind, SNG, SCI, ISO, CHT, ECH and 200/10227/9276/6677 m wind levels), DOM_SECOND empty]
 EXTRACTLIST_MULTHEIGHT="string": keywords extracted as a function of height. [current: "SNG_STAR_SUP CHT_STAR_SUP"]
 PVLIST_DOM_FIRST,
 PVLIST_DOM_SECOND="string": diaprog keywords for vertical profile extraction. [current: both empty]
 STARTPV_I[0], STARTPV_J[0]: grid point for the DOM_FIRST-related vertical profile. [current: 61,61]
 STARTPV_I[1], STARTPV_J[1]: grid point for the DOM_SECOND-related vertical profile. [current: 51,51]
 CVLIST_DOM_FIRST,
 CVLIST_DOM_SECOND="string": diaprog keywords for vertical cuts. [current: both empty]
 STARTCV_I/J[0], ENDCV_I/J[0]: start/end grid points of the DOM_FIRST-related vertical cut. [current: (2,61) to (121,61)]
 STARTCV_I/J[1], ENDCV_I/J[1]: start/end grid points of the DOM_SECOND-related vertical cut. [current: (2,51) to (101,51)]
 NUMLEVELDIAG=integer: number of nesting levels on which DIAG runs. [current: 1]
 DIAGLEVELARRAY[n] : which nesting level DIAG runs on, for each of the NUMLEVELDIAG entries. [current: [0]=\$DOM_FIRST, [1]=\$DOM_SECOND (only [0] used since NUMLEVELDIAG=1)]
 DIAGINFHEIGHTS[n],
 DIAGSUPHEIGHTS[n] : space-separated lists of lower/upper integration heights for DIAGLEVELARRAY[n]; every INF-SUP combination is computed. [current: [0]="\$PLOTBASEH1 \$PLOTBASEH2" / "\$PLOTTOPH1 \$PLOTTOPH2"]
 DOMP1=1, DOMP2=2 : fixed identifiers for the two extraction-point domains; used elsewhere in `plot.sh`'s domain selection logic and should not be changed.

4.6 times.var

`times.var` configures the date being processed, ephemerides-related timing and the plot time window. A large part of the file (ephemerides, seasonal plot-limit computation) is scripted and should not be edited directly.

How the date is actually set. At the top of the file, `YEAR/MONTH/DAY` default to the system date (`$(date +%Y)` etc.), but the checked-in file immediately overrides them with a hand-set date a few lines below (a leftover from the last manual/debug run). **Because this codebase has no external process that refreshes `times.var` before each run**, whichever of the two is active when `run.sh` sources this file is what gets used:

- to process “today” automatically on every scheduled run, comment out (or remove) the hand-set override so the system-date lines take effect;
- to reprocess a specific past night, or to pin the date for a manual test run, edit the hand-set override directly.

There is a second file, `times.var_sequential`, which is a template of `times.var` using the placeholders `SET_THE_DATE_YEAR/SET_THE_DATE_MONTH/SET_THE_DATE_DAY/ SET_THE_UPLOADFIGS` in place of literal values. Nothing in this codebase currently reads or substitutes into this template – it is not sourced by `run.sh`, and no other script in `bin/` references it. It is a leftover from a deployment where an external, per-night wrapper script generated `times.var` from it; on this single-configuration codebase either ignore it, or use it as the starting point for your own such wrapper if you want one, substituting the four placeholders and copying the result over `times.var` before invoking `run.sh`.

4.6.1 Date and time

```
YEAR=yyyy, MONTH=mm, DAY=dd : date (UT) of the start of the forecast (see above for
how
                                this is actually set).
FRCSTDATECOMPUTED=hh: UT hour at which the forecast used for initialization was
computed.
                                [current: 12]
FORWARDFRCSTDATEPLUS=hours: how far in the future, from FRCSTDATECOMPUTED, the first
                                forecast file is valid for. [current: 12]
DATEFIGLT=0/1          : if 1, the date used to label figures is the local-time start of
the
                                night; if 0, it is the UT date of the end of the night. [current:
                                1]
HOUR_SHIFT_PLOT=hours: shift of the plot time axis with respect to UT midnight, to
align
                                with the simulation start. [current: 0]
UTOFFSET=hours         : local time offset from UT at the site. [current: -7]
HOUR_LIST="string"     : space-separated list of forcing hours; matching initialization
                                files must exist for each. [current: "00 06 12 18"]
TIMESEND=hh            : UT hour by which the init files are expected to have arrived.
                                [current: 18]
TIMELIMIT=seconds      : how long to keep waiting/retrying past TIMESEND before aborting
                                with an error. [current: 10800]
EXITSEC=seconds        : interval between Meso-NH output times; must match OUTTIMESTEP in
run.var. [current: 3600]
```

(EXITMIN is simply EXITSEC/60, computed automatically.)

4.6.2 Plot parameters

PLOTSTARTT/PLOTSTARTEXTIT, PLOTFIRSTT/PLOTFIRSTEXIT, PLOTSECONDT/PLOTSECONDEXIT, PLOTENDT/PLOTENDEXIT	: default (non-seasonal) plot time window, in minutes from simulation start and corresponding Meso-NH output index; overwritten by the seasonal computation below when SEASONSLOT=1 (the default).
FIRSTMONTH=mm, LASTMONTH=mm	: calendar months bounding "summer" for the seasonal plot window. [current: 04, 09]
EMISPHERE=1/-1	: 1 for the northern hemisphere, -1 for the southern. [current: 1]
SEASONSLOT=0/1	: if 1 (default), the plot time window is computed each night from sunset/sunrise/dusk/dawn (via the Sunset.py / Dusk.py / Dawn.py / Sunrise.py helpers) instead of using the fixed values above. [current: 1]
UPLOADFIGS=0/1	: tells backup.sh whether to upload all figures (1) or only the wind-at-ground figures, and only if WINDSPEEDHIGHLIM is exceeded (0) -- see Section~\ref{sec:usage}. A static per-deployment setting here (there is no longer anything that overrides it per run). [current: 1]

4.7 backup.var

backup.var configures what gets preserved/uploaded at the end of a run and where.

4.7.1 Upload / backup switches

UPLOAD=0/1	: upload figures/ASCII outputs to the remote server(s). [current: 1]
ARUPLOAD=0/1	: rsync the AUTOREGRESSION hand-off directory to the short-timescale component's host(s) at the end of run.sh, if that component is in use on this deployment. [current: 1]
BACKUP=0/1	: perform the local backup of output files. [current: 1]
BACKUPFIGURES=0/1	: keep a local mirror of the delivered figures. [current: 1]

4.7.2 Backup parameters

TAIL_DIR="string"	: suffix appended to the per-night backup directory name. The checked-in value is a short project tag inherited from this codebase's origin; edit to identify your own deployment.
BACKUP_DIR="string"	: root of the local ASCII/plot backup. [current: "\${HOME}/BACKUP_OUTPUT"]
BACKUP_FIGURES_DIR="string"	: root of the local figures mirror. [current: "\${HOME}/BACKUP_FIGURES"]
BACKUP_REAL, BACKUP_RUN, BACKUP_DIAG=0/1	: whether to keep, respectively, the PREP_REAL / EXE(RUN) / DIAG binary output in the per-night backup (rather than discarding it). [current: 0, 0, 0]
BACKUP_MNHDAT=0/1	: keep the ASCII (.dat, FICVAL, AUTOREGRESSION) postprocessing output. [current: 1]
BACKUP_POSTPROC=0/1	: keep the ASCII+PLOT postprocessing output tree. [current: 1]
BACKUP_LOG=0/1	: keep the run's log directory (required for backup.sh to be able to relocate its own log paths correctly at the end of the run).

[current: 1]

The corresponding `BACKUP_*.NAME` variables are the actual destination paths, all built from `BACKUP_BASE_NAME` (itself `BACKUP_DIR/<date><TAIL_DIR>_<N>DOM_<MESONH version>`); they are not normally edited directly.

4.7.3 Remote host parameters

`DAYSTARTBEFORE=integer (negative):` how many days before the forecast date the simulation started; used to locate the right backup sub-directory.
[current: -1]

`REMOTE_USER="string", REMOTE_HOST="string":` account/host of the primary remote output server. Placeholder values in the checked-in file -- must be filled in for your own deployment.

`REMOTE_USER_BK="string", REMOTE_HOST_BK="string":` account/host of the redundant backup remote server. Also placeholders in the checked-in file.

`REMOTE_PATH_ARCHIVE="string", REMOTE_PATH_OUTPUTS_GROUND="string":` upload paths on the primary server for figures and ground-parameter ASCII output respectively (trailing slash required). The checked-in values still point at a path tree named after this codebase's origin; edit to match your own server layout.

`REMOTE_PATH_ARCHIVE_BK="string", REMOTE_PATH_OUTPUTS_GROUND_BK="string":` matching upload paths on the backup server.

`REMOTE_AR="string", REMOTE_AR_BK="string":` address(es) of the short-timescale component's hand-off host(s), if that component is deployed; the AUTOREGRESSION directory is rsynced there at the end of `run.sh`. Placeholders in the checked-in file.

`REMOTE_PATH_AUTOREG="string":` remote path (trailing slash) for that hand-off rsync.

4.8 filedef.var

`filedef.var` is mostly scripted logic rather than user-facing parameters: it implements the search for the day's PGD files, and the wait/retrieval/validation of the initialization files described in Section 2 (matching against `HOURLIST` from `times.var`, retrying from the backup init server after `TIMELIMIT`, verifying MD5 checksums, merging orography via `grib_copy`). It must be sourced after `run.var` and `times.var`, on which its logic depends.

Despite the file's own header warning not to modify anything past a certain point, two variables here **do sometimes legitimately need editing**, because they encode the forecast-data provider's current naming convention for the delivered files rather than anything internal to FAST:

`BASEINITNAME="string" : expected base filename of the day's raw forecast files.`
[current: "D2D\${MONTH}\${DAY}\${FRCSTDATECOMPUTED}00"]

`BASEANAFILE="string" : expected filename of the initial analysis file (the one`
`grib_copy`
`merges orography from). Built by repeating the BASEINITNAME`
`date/hour tokens and appending a trailing "001" segment.`
[current starts: "D2D\${MONTH}\${DAY}\${FRCSTDATECOMPUTED}00..."]

If the provider ever changes how it names the files it delivers, these two variables are the ones to update to match; the rest of the search/retrieval logic is built on top of them and should not need to change.

TESTRUN=1 behaviour. When `TESTRUN=1` (`globals.var`), all of the search/wait/ retrieval logic above is skipped outright, and every element of `FILE_ANA_ARRAY` (the array of init file paths consumed by the rest of `run.sh`) is set to a non-existent placeholder (`${INIDIR}/foo`) rather than a real file. For a `TESTRUN=1` run to actually work, the commented-out block near the end of the file must be edited by hand:

```
#####
#EDIT THIS TO OVERWRITE GRIB FILES SELECTION FOR DEBUG PURPOSES
#####
```

Uncommenting and filling in the `FILE_ANA_ARRAY[...]=...` lines there (a few worked examples, commented out, are already present as a starting point) is what supplies real, existing file paths in place of the placeholder – this is not an optional convenience for advanced debugging, it is the only way `TESTRUN=1` produces a working run at all.

5 EXECUTABLES REFERENCE

This section describes each script individually. Section 2 already covers how they fit together into the nightly workflow; here the focus is on each script’s own inputs, outputs and notable behaviour.

5.1 bin/run.sh – main nightly executable

Entry point of a single night’s simulation; invoked with no arguments, either by cron or manually (Section 2). It locates its own configuration via `conf.d/globals.var`, relative to its own path, so it must be run from within `bin/`. On start it kills any leftover Meso-NH-related processes from a previous, possibly stuck run (`pkill` on the executable names from `globals.var`), rotates the log directory, then runs the full phase sequence described in Section 2 (`prep_list.sh`, `filedef.var`, `PREP_REAL`, `EXE`, `postproc.sh`, `plot_python.sh`, compression, staging, `backup.sh`, notification, hand-off). Exit status is 0 on success; any hard failure calls the `error` function (log + alert mail + `exit 1`). Individual phases can be disabled with the `NOPREP/NORUN/NODIAG/NOFICVAL/NOPLOT` debug switches in `globals.var` (Section 4).

5.2 bin/prep_list.sh – generate reference legacy-style scripts

Invoked by `run.sh` as a separate sub-process (`env bash -c`) before the `PREP_REAL` phase, unless `PREPLISTTRUE=0`. Its header comment (“script used to generate runscripts with all the namelists inside, Lascaux&Masciadri style”) is accurate and matches what most of the script actually does: it generates **two self-contained, standalone bash scripts**, written into `WORKDIR`:

```
1.PREP_EXP_REAL_<version>_<site>_<N>MOD_<date>_<levels>lev_<tail> : reproduces the
                                                                    PREP_REAL/SPAWNING step
2.PREP_EXP_RUN_<version>_<site>_<N>MOD_<date>_<levels>lev_<tail> : reproduces the
                                                                    EXE (Meso-NH run) step
```

Each is fully self-contained – exported shell variables, the relevant namelists embedded via here-docs, the exact `mpirun` invocation – written in the style of the pre-FAST Lascaux & Masciadri Meso-NH workflow scripts. They are meant as **reference material for a user unfamiliar with FAST** who wants to see, or manually reproduce, what happens at the plain Meso-NH level; `run.sh`’s own `PREP_REAL/EXE` phases are separate, inline implementations and do **not** use these generated scripts at all.

Generating them requires knowing which init files are available, so `prep_list.sh` does source `globals.var`, `times.var`, `run.var`, `filedef.var` and `backup.var` (with `GENERATELIST=1`, which suppresses the verbose logging those files would otherwise produce). Because it runs in a separate shell, none of the variables it resolves survive back into `run.sh`. At the end, `prep_list.sh` deletes the REAL/RUN staging directories it used while generating the scripts (`rm -rf`); only the two generated scripts are left behind in `WORKDIR`.

If invoked interactively (with any command-line argument) it will instead prompt for YEAR/MONTH/DAY/DAYB rather than reading them from `times.var`.

5.3 bin/postproc.sh – DIAG/FICVAL extraction

Sourced (not executed as a sub-process) by `run.sh` right after the EXE phase, so it shares `run.sh`'s environment. Unless `NODIAG=1`, it loops over every Meso-NH output (`.lfi`) file, and for each nesting level listed in `DIAGLEVELARRAY` runs the `DIAG` executable and `conv2dia/diaprogram` for every combination of heights in `DIAGINFHEIGHTS/DIAGSUPHEIGHTS` (`run.var`, Section 4), producing the derived astroclimatic parameters. Unless `NOFICVAL=1`, it also runs the FICVAL extraction (vertical profiles/cuts and 2D ground maps) for the keywords configured in `EXTRACTLIST_DOM_FIRST/EXTRACTLIST_DOM_SECOND/PVLIST_*/CVLIST_*`. Output goes to `DIAG_MNH/ASCII_DIR` (Section 2).

5.4 bin/plot_python.sh – plotting

Sourced by `run.sh` after `postproc.sh`, unless `NOPLOT=1`. Drives the Python scripts under `bin/conf.d/Utils/Plot_python/` (Section 4) to turn the ASCII output of `postproc.sh` into the delivered figures under `PLOT_DIR`, and computes `WINDAVG` (the K=4 average wind speed) and the derived `WINDAVGHIGH` flag against `WINDSPEEDHIGHLIM` (`run.var`), used by `backup.sh` as described in Section 2. Uses ImageMagick's `convert` for at least one figure format conversion step.

It computes the full parameter suite on `DOM_FIRST`: CN2 (both a “new-algorithm” wind-gradient-corrected product and, in parallel, the `_OS18` alternative product, Section 4), seeing, isoplanatic angle and coherence time, potential and absolute temperature, relative humidity, precipitable water vapour, plus wind at several fixed levels and vertical profiles, and 2D ground maps for most of the above. Among the wind products, it also computes statistics over three fixed height windows tied to specific instrument requirements at the deployment site (script-internal suffixes `_linc_low/_linc_high`, `_flao`, `_argos` – currently 30–415 m, 5072–9127 m, 30–20000 m and 30–560 m respectively); edit the ranges and labels directly in the script if a different deployment's instruments need different windows. It unconditionally copies its ground-parameter results (seeing/wind/temperature/RH/PWV) into `AUTOREGRESSION_DIR` and `AUTOREGRESSION_STORE_DIR`, ready for the short-timescale hand-off described in Section 2, if that component is deployed.

5.5 bin/backup.sh – upload and local backup

Sourced by `run.sh` at the very end of a successful run, unless `BACKUPLATER=1` (in which case it must be invoked separately). Reads `backup.var` (Section 4) to decide, per output category, whether to rsync it to the primary and backup remote servers (`REMOTE_HOST/REMOTE_HOST_BK`), and/or keep a local copy under `BACKUP_DIR` (or, for figures, `BACKUP_FIGURES_DIR`). When `UPLOADFIGS=0` (`times.var`) it normally skips the figure upload entirely, *except* that it still uploads the near-ground wind figures/ASCII if `WINDAVGHIGH=1` (set by `plot_python.sh`) – the self-contained high-wind exception described in Section 2. Can also be run stand-alone (`./backup.sh`) for a

previously-staged run, in which case it detects it is not being sourced from `run.sh` and sources `globals.var`, `times.var`, `run.var` and `backup.var` itself first.

5.6 bin/pgdgen.sh – PGD (orography) generation

Stand-alone script, not called by `run.sh`; run manually at installation time and again only if the domain geometry or topography source files change (Section 3). Sources `globals.var` and `pgd.var`, checks/creates the working directories, then generates the PGD files for every nesting level (outer domain plus each nested level, per `NESTING`) via the internal `prep_list_pgd` function (which in turn drives Meso-NH’s `PREP_PGD/` `PREP_NEST_PGD` executables using the `NLPGDFIRST/NLPGDNEST/` `NLPGDPREPNEST` namelist templates). On completion it moves its own log files into `PGDDIR`. Takes no arguments; progress and errors go to `LOGPGDFILE`.

5.7 Benchmarking helpers (not part of nightly operation)

`bin/Run_lista.sh`, `bin/crossc2d.py` and `bin/multi.py` are used together to replay a list of past initialization dates (read from a file named `LISTAgrib`) purely to measure CPU/timing performance. `Run_lista.sh` builds a `RUN_ALL.sh` launcher from that list and copies the two Python helpers into a scratch `CPUTEST/` directory. None of the three are invoked anywhere in the operational workflow (cron or otherwise) and can be ignored for normal use.

6 GROUND-PARAMETER OUTPUT FILE FORMAT

This section documents the format of the per-night ground-parameter ASCII file produced by this deployment: written under `FINALOUTDIR` (Section 4) and uploaded by `backup.sh` to the remote output path configured in `backup.var` (Section 4). This is a single, purely deterministic file per night, with no probabilistic breakdown; other deployments, delivering to a different downstream consumer with different requirements, may need a different output format entirely – the format below is specific to this configuration’s `plot_python.sh`, not a property of the automation framework itself.

6.1 File naming and header

Each file is named after the date of the relevant night in UT (`YYYYMMDD`), counting from 00:00 UT. Every file starts with a 5-line header giving the forecast start date and the dusk/dawn times for that night, followed by the column list:

```
## OUTPUTS ##
## DATE_MST(YYYYMMDD)=20200413
## DUSK (EPOCH)=1586834340
## DAWN (EPOCH)=1586863209
## TIME (EPOCH), TIME FROM SIMULATION START(min, UT), TEMPERATURE (K), TEMPERATURE SD
(K),
WIND SPEED (m/s), WIND SPEED SD (m/s), WIND DIRECTION (deg), WIND DIRECTION SD
(deg),
RELATIVE HUMIDITY (%), RELATIVE HUMIDITY SD (%), TOTAL SEEING (arcsec),
TOTAL SEEING SD (arcsec) ##
```

(DUSK/DAWN are given in Unix epoch seconds; the last header line has been wrapped above only for page width – it is a single line in the actual file.)

6.2 Processing

- Values are processed with a **1-hour moving average** and resampled every **20 minutes**. The reported standard deviation (“SD”) is computed over the 20-minute sampling window *before* the moving average is applied, not over the smoothed series.
- Atmospheric values (temperature, wind, relative humidity) are taken at model level **K=4**. On this deployment that level was chosen to match the local weather station’s reference height (currently configured as an example at **38–62 m above ground level**); adjust to your own site’s instrumentation height. This is the same K=4 level used by the WINDSPEEDHIGHLIM logic (Section 2).
- Total seeing is integrated over **20–2000 m above ground level** and calibrated against a local Differential Image Motion Monitor (DIMM).

6.3 Columns

12 space-separated columns:

#	Column
1	Timestamp, Unix epoch (UT)
2	Time from the start of the forecast (00:00 UT), in minutes
3	Temperature, K
4	Temperature SD, K
5	Wind speed, m/s
6	Wind speed SD, m/s
7	Wind direction (wind blowing from North = 0°, from East = 90°), degrees
8	Wind direction SD, degrees
9	Relative humidity, %
10	Relative humidity SD, %
11	Total seeing, arcsec
12	Total seeing SD, arcsec